Pyrmit – benchmark report

Feasibility questions on Swedish planning law: tool-using agent vs. classic RAG. Model: GPT-6 Luna via OpenRouter. Runs: 2026-10-07 to 2026-10-17; report generated on 2026-10-25. How the system works and how each benchmark set is built are explained in the **technical report**; this report gives the test conditions and the results.

Summary

- Compared: a **tool-using agent** and a **classic RAG** (one search, one answer), same model and same corpus, on three sets: 14 hand-written case studies, 50 cases from real rulings of the Land and Environment Court of Appeal (MÖD) and 20 questions on a property of a real plan map.
- **The agent wins on everything that requires finding the deciding rule**, and the gap grows with difficulty. Key points: 0.77 vs 0.59 (14 cases), 0.48 vs 0.21 (MÖD), 0.67 vs 0.31 (plankarta). Expected evidence seen: 0.88 / 0.70 / 1.00 vs 0.60 / 0.12 / 0.47.
- **The RAG is about twice as fast and four times cheaper**: 16–24 s per question (first word after 11–18 s), $0.0020 per question, against 31–107 s and $0.0071–$0.0110 for the agent. Its grounding is as high or higher (0.81–0.90): it says less, but what it says is in its sources.
- **Verdict**: all systems score ≥ 0.96 on the 14 cases; on MÖD the agent gets 0.70–0.75 against 0.60 for the RAG, on the plan maps 0.62 against 0.35. The agent's weak spot is still the projects the court refused (39 %–59 % correct): it leans towards “feasible with conditions”.
- **Map reading matters** on the plan-map cases: without the map tools the agent's verdict drops from 0.62 to 0.47 and its key points from 0.67 to 0.54, but the map adds about 54 s per question.
- **Exploration effort is the agent setting that matters**: *low* → *medium* on MÖD raises evidence 0.53 → 0.70, grounding 0.69 → 0.79 and key points 0.43 → 0.48, for +17 s and 2.4× the cost; a *high* final answer brings nothing measurable.
- **Beyond the verdict** (38 cases asking to explain a procedure, propose where a project could go, or develop a project idea; section 2.7): agent vs RAG key points 0.69 vs 0.63 (explain), 0.53 vs 0.41 (propose), 0.70 vs 0.47 (develop). The agent recognises the kind of question (mode right 0.92 / 0.42 → 1.00 after the propose fix / 0.92), and the mode prompt leaves the verdict sets unchanged (section 2.8).

1. Test conditions

1.1 Systems compared

System	What it does	Settings tested
Classic RAG	One rewrite call (English question + Swedish search query, effort low) → one dense search on the Swedish query (arctic-embed-l-v2, municipality filter detected from the question) → the top 10 statute sections and top 10 municipal passages → one answer call, streamed. It cannot open a document, list plans or look at a map.	Answer effort *medium*, streamed (the history table also shows an earlier *high*, non-streamed run)
Agent	Tool loop: exploration turns (search, grep, read a document, statute section, plan registry and plan rules, map crop around a property, PDF page image), tools of a turn run in parallel, then `answer_ready` and one streamed final answer.	Exploration effort *low* / *medium* × answer effort *medium* / *high*; on the plankarta cases, ablations without the map tools

Both answer in the same format (verdict, key rules with [c:id] citations, process, uncertainties) and search the same corpus: the full disk store (15 statutes, the crawled websites of the municipalities, 8,252 indexed detailed plans), as of 2026-10-22. The larger crawl of 2026-10-23 (about five times the documents) is not benchmarked yet.

1.2 Benchmark sets

Set	Origin	Cases run	What it tests	Reference
14 case studies	Hand-written from real documents of 5 municipalities (detailed plans of 3 real projects, municipal guidance pages, statutes) and questions from user feedback	14 (all)	Combining national and local rules on common projects	Accepted verdicts, 3–5 key points, expected evidence (statute section or exact municipal excerpt)
MÖD rulings	Real appeals decided by the Land and Environment Court of Appeal (746 rulings since 2011, 390 on our topics); the facts become the applicant's question, asked before the decision	50 of 260 (stratified by outcome, fixed seed)	Legal judgement on real disputes; verdict against the real outcome	Outcome → accepted verdicts; court's reasons → key points; statutes applied → evidence (current wording)
Plankarta	Real plan maps (in force or adopted) of 18 municipalities, a property number printed on the map; the project is designed so that its feasibility depends on the zone rules	20 (high-confidence references)	Reading the map: which zone the property is in, which codes apply	Codes and values read on the map crop + legend by a vision model; evidence = the plan map document
Explain / propose / develop	10 cases written by hand from real documents (Vallentuna, Norrtälje, Uppsala, Gotland; questions from user feedback and the ReGen Villages vision) + 28 drafted by MiMo v2.6 Flash from corpus documents (municipal guidance pages, the growth and rural passages of comprehensive plans, statutes), in other municipalities	all	Explaining a procedure; proposing where a project could go (places the comprehensive plan designates); developing a project idea (obstacles, ways to make it work, what to ask the user)	Key points with a verbatim quote each (checked automatically); propose: the designated places (anchors); develop: points labelled Obstacle / Lever / Ask; no verdict

1.3 Protocol and scoring

- Same model for the systems and the judges (GPT-6 Luna via OpenRouter); judge effort medium. The explain / propose / develop sets are judged by MiMo v2.6 Flash, another model family, which also drafted their generated cases.
- **Judge check**: 60 stored answers (14 cases and MÖD, agent and RAG) graded again by xiaomi/mimo-v2.6-flash: same verdict class in 95 % (kappa on “verdict correct” 0.89), key points 0.44 vs 0.43 (correlation 0.90, mean gap 0.08). The second judge counts fewer unsupported claims (0.8 vs 2.0): Luna is stricter, not lenient towards its own answers.
- Each configuration runs 2 passes or more (n = answers = cases × passes); cases run 3 in parallel, so latencies include some contention. Answers are streamed: “1st token” is the time until the first word of the answer.
- Results are written per answer (`data/agentic/bench-*.json`) with the full answer, the tool calls and the judge's comment.

Measure	Definition	How
Verdict	The answer's verdict (feasible / with conditions / not feasible / unclear) is among the accepted verdicts	LLM judge
Key points	Each expected point 0 (missing or contradicted), 1 (partial), 2 (clear); scaled to 0–1	LLM judge
Evidence seen	Share of the expected evidence the system *actually had in front of it* (retrieved passages for the RAG, tool outputs for the agent)	Automatic, no LLM
Evidence cited	Share of the expected evidence the answer *cites*: did the system recognise the decisive passage among what it read? The gap with "evidence seen" is the passages read but not used	Automatic, no LLM
Passages seen	Distinct passages the system had in front of it per question: the reading volume. A precision ("relevant / read") is not measurable: the labels list the decisive passages, not every useful one	Automatic
Grounding	Share of the answer's factual claims supported by the text of the sources it cites	Second LLM judge, full source text
Latency, 1st token, cost	Total time, time to the first answer word, OpenRouter cost of the system's calls (judges excluded)	Measured

> **Limits.** The judge is the system's own model: cheap but possibly lenient, so its scores *compare* settings rather than give absolute quality. MÖD key points and plankarta references are written by an LLM (from the ruling, from the map); no human review yet. A difference below ~0.05 between configurations is within the pass-to-pass noise (section 2.6). **Scope**: the first three sets check the feasibility of a defined project at a known place; the explain / propose / develop sets (section 2.7) add explaining, proposing places and developing an idea, in a single turn; multi-turn guidance is still not measured (technical report, section 7.8).

2. Results

2.1 Overview: classic RAG vs. agent (default setting, medium / medium)

Case set	System	n	Verdict	Key points	Evidence seen	Evidence cited	Grounding	Passages seen	Latency	1st token	Cost / question
14 cases	RAG	28	1.00	0.59	0.60	0.48	0.90	20	16 s	11 s	$0.0017
	Agent (medium / medium)	28	1.00	0.77	0.88	0.54	0.89	69	31 s	24 s	$0.0071
MÖD	RAG	100	0.60	0.21	0.12	0.11	0.88	20	23 s	17 s	$0.0020
	Agent (medium / medium)	200	0.70	0.48	0.70	0.40	0.79	78	43 s	35 s	$0.0086
plankarta	RAG	40	0.35	0.31	0.47	0.28	0.81	20	24 s	18 s	$0.0020
	Agent (medium / medium)	40	0.62	0.67	1.00	0.90	0.68	66	107 s	99 s	$0.0110

The agent reads 38–80 passages per question through 12–27 tool calls, each with its own query; the RAG reads the 20 passages of a single search. The number is not the point (a case has 1 to 2 expected passages): the RAG's single query, written from the user's facts, rarely lands on the statute section or the plan map that decides the case, whereas the agent first works out which rule applies, then looks it up. Of what it sees, the agent cites about three quarters of the expected evidence (evidence cited vs seen); the RAG cites most of what it finds, but outside the 14 case studies it finds little.

2.2 14 case studies

History (5-municipality sub-corpus, successive versions):

Version	Verdict	Key points	Evidence	Grounding	Latency	1st token	Cost
Classic RAG (1 search, high-effort answer, not streamed)	1.00	0.56	0.58	0.84	30 s	–	$0.0021
Agent v1 (high effort throughout)	1.00	0.75	0.86	0.68	64 s	–	$0.0057
Agent v2 (citeable sources, medium effort)	1.00	0.77	0.96	0.86	39 s	39 s	$0.0042
Agent v3 (low exploration, medium answer)	1.00	0.68	0.73	0.82	19 s	13 s	$0.0025
Agent v4 (v3 + read the key municipal document)	1.00	0.65	0.84	0.84	23 s	17 s	$0.0036

v1 → v2: citeable identifiers in every tool output and a "no claim without a source read" rule (grounding 0.68 → 0.86). v2 → v3/v4: parallel tools, `answer_ready`, streamed answer and *low* exploration effort: latency halved, lower quality. The RAG of this table ran with a *high*, non-streamed answer; it is re-measured below at *medium*, streamed.

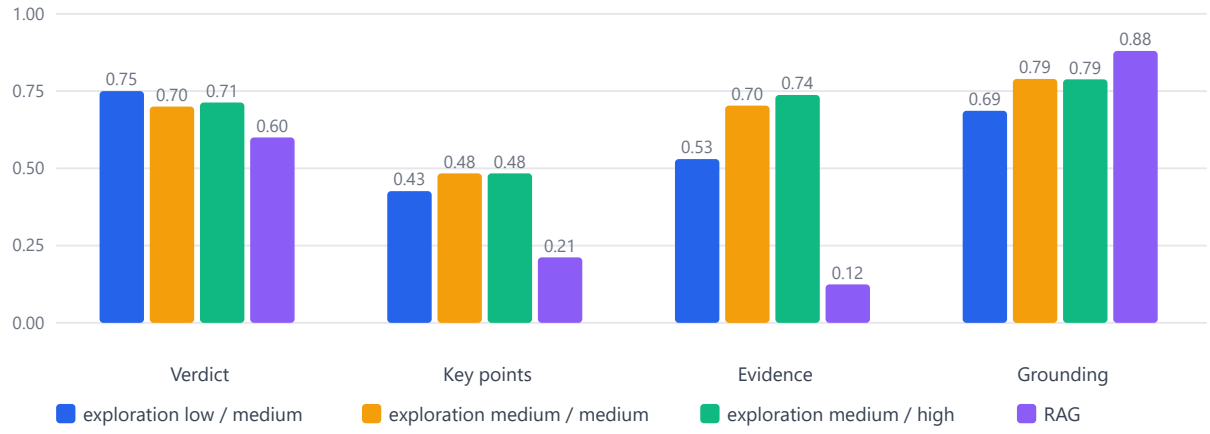
Effort matrix (full corpus on disk):

Configuration	n	Verdict	Key points	Evidence seen	Evidence cited	Grounding	Passages seen	Tool calls	Latency	1st token	Cost / question
Agent, exploration low / answer medium	28	0.96	0.70	0.84	0.57	0.85	38	12.0	28 s	22 s	$0.0036
Agent, exploration medium / answer medium	28	**1.00**	**0.77**	**0.88**	0.54	0.89	69	20.2	31 s	24 s	$0.0071
Agent, exploration medium / answer high	56	0.98	0.76	0.87	**0.60**	0.85	63	20.2	38 s	31 s	$0.0069
Classic RAG (medium answer)	28	**1.00**	0.59	0.60	0.48	**0.90**	20	0.0	16 s	11 s	$0.0017

2.3 MÖD cases (50 sampled rulings)

Configuration	n	Verdict	Key points	Evidence seen	Evidence cited	Grounding	Passages seen	Tool calls	Latency	1st token	Cost / question
Agent, exploration low / answer medium	100	**0.75**	0.43	0.53	0.28	0.69	40	12.3	26 s	19 s	$0.0036
Agent, exploration medium / answer medium	200	0.70	0.48	0.70	0.40	0.79	78	26.4	43 s	35 s	$0.0086
Agent, exploration medium / answer high	150	0.71	**0.48**	**0.74**	**0.40**	0.79	80	26.7	51 s	42 s	$0.0092
Classic RAG (medium answer)	100	0.60	0.21	0.12	0.11	**0.88**	20	0.0	23 s	17 s	$0.0020

Best value of each column in green. n = number of answers (cases × passes).



Case by case on the 50 MÖD cases, `medium` exploration beats `low` on 25 cases, does worse on 3, and ties (±0.05) on 22.

Verdict by actual outcome (in brackets: number of answers; "open outcome" = remitted or fact-dependent decision, accepted verdicts "with conditions" / "unclear"):

Configuration	Project refused (correct verdict)	Project granted	Open outcome	On refusals, answers "with conditions"
Agent, exploration low / answer medium	0.59 (32)	0.74 (34)	0.91 (34)	0.28
Agent, exploration medium / answer medium	0.39 (64)	0.79 (68)	0.90 (68)	0.44
Agent, exploration medium / answer high	0.50 (48)	0.73 (51)	0.90 (51)	0.38
Classic RAG (medium answer)	0.53 (32)	0.32 (34)	0.94 (34)	0.03

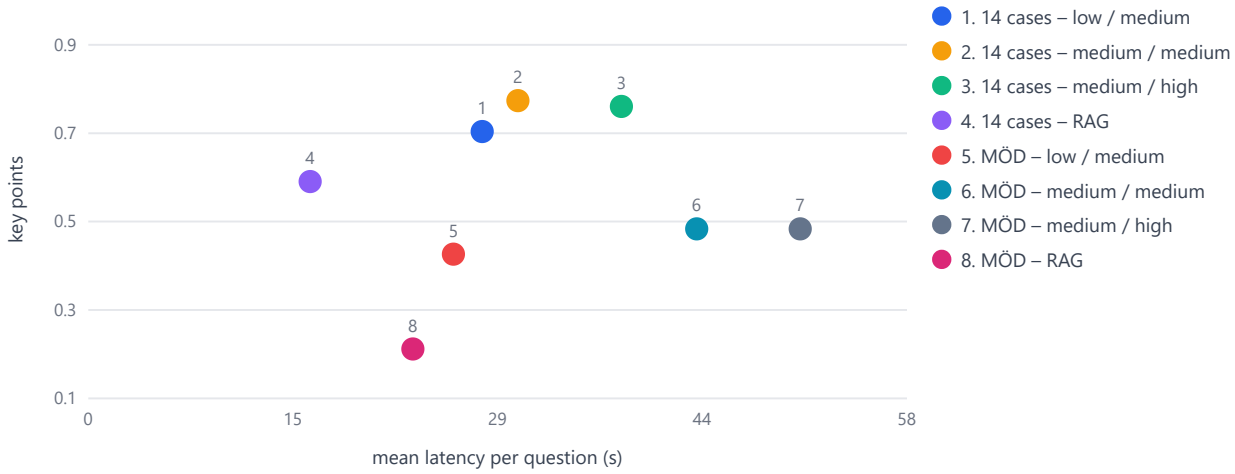
2.4 Plankarta cases (map reading)

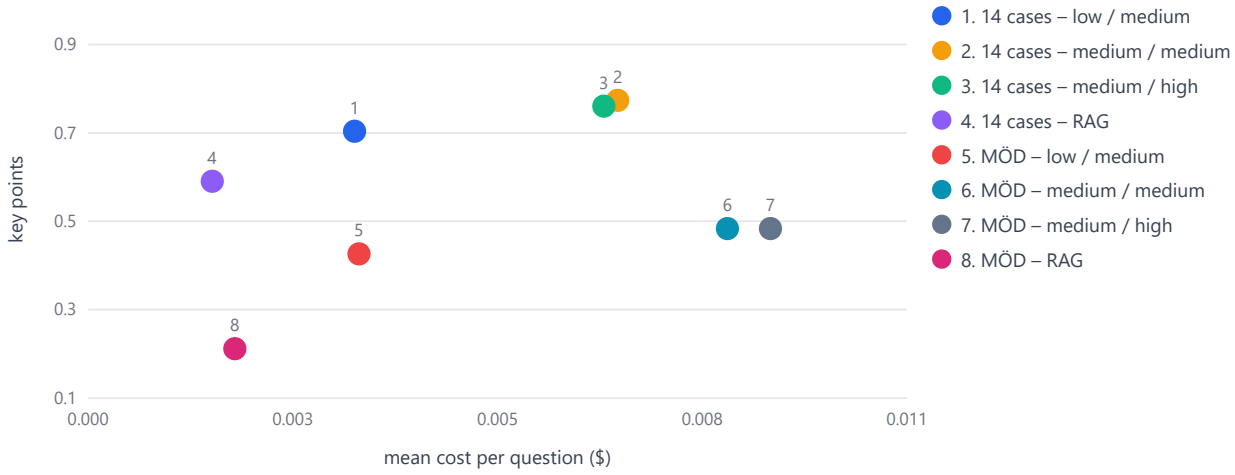
Configuration	n	Verdict	Key points	Evidence seen	Evidence cited	Grounding	Passages seen	Tool calls	Map views	Latency	1st token	Cost / question
Agent, exploration medium / answer medium	40	**0.62**	**0.67**	**1.00**	**0.90**	0.68	66	25.3	2.3	107 s	99 s	$0.0110
Agent, exploration medium / answer medium – no view_map_area	40	0.60	0.57	0.97	0.82	0.82	77	26.6	1.7	77 s	70 s	$0.0102
Agent, exploration medium / answer medium – no map reading at all	40	0.47	0.54	**1.00**	0.82	**0.84**	75	24.1	0.0	53 s	45 s	$0.0082
Classic RAG (medium answer)	40	0.35	0.31	0.47	0.28	0.81	20	0.0	0.0	24 s	18 s	$0.0020

Map views = images of the plan map the agent looked at per question (`view_map_area`, `view_pdf_page`). Evidence = the plan map of the case was among the passages seen.

With both map tools the agent gets the verdict right in 0.62 of the answers and covers 0.67 of the key points; without `view_map_area` (whole-page images only) 0.60 / 0.57; with no map at all 0.47 / 0.54: it has to guess the zone from the text. The map views make this set slow (107 s per question). Grounding is lower with the map (0.68 vs 0.84), probably because what is read on the image (which zone the property is in) cannot be checked by the grounding judge, which only gets text. The classic RAG, with one search and no map, has the plan map of the case among its passages in 0.47 of the answers and gets the verdict right in 0.35.

2.5 Quality / speed / cost trade-off (14 cases and MÖD)





2.6 Variance between passes

- Agent, exploration low / answer medium (14 cases): key points per pass 0.74 / 0.67
- Agent, exploration medium / answer medium (14 cases): key points per pass 0.75 / 0.79
- Agent, exploration medium / answer high (14 cases): key points per pass 0.77 / 0.78 / 0.71 / 0.78
- Classic RAG (medium answer) (14 cases): key points per pass 0.58 / 0.60
- Agent, exploration low / answer medium (MÖD): key points per pass 0.43 / 0.43
- Agent, exploration medium / answer medium (MÖD): key points per pass 0.50 / 0.50 / 0.46 / 0.47
- Agent, exploration medium / answer high (MÖD): key points per pass 0.49 / 0.47 / 0.49
- Classic RAG (medium answer) (MÖD): key points per pass 0.21 / 0.21
- Agent, exploration medium / answer medium (plankarta): key points per pass 0.68 / 0.66
- Agent, exploration medium / answer medium – no view_map_area (plankarta): key points per pass 0.56 / 0.58

- Agent, exploration medium / answer medium – no map reading at all (plankarta): key points per pass 0.51 / 0.57
- Classic RAG (medium answer) (plankarta): key points per pass 0.37 / 0.25

Typical gap between two identical passes: a few hundredths; a difference below ~0.05 between configurations is not significant.

2.7 Tasks beyond the verdict: explain, propose, develop

From 2026-10-19 the agent first classifies the question (`answer_ready(mode)`: verdict, explain, propose, develop) and answers in that mode's format; the RAG gets the same format instructions in one prompt. Judge: MiMo v2.6 Flash. "Mode right" = the agent chose the case's task. Places = share of the plan-designated places the answer names (string match, at least `min_anchors` needed); options sourced = share of the proposed options the cited sources support (judge). Latencies of these runs include the query embedding on the CPU (ONNX), not the GPU.

Task	System	n	Key points	Evidence seen	Evidence cited	Unsupported	Mode right	Task-specific	Latency	Cost / question
explain	RAG	13	0.63	0.83	0.59	0.9	–	–	13 s	$0.0013
	Agent	13	0.69	1.00	0.84	0.2	0.92	–	33 s	$0.0042
propose	RAG	12	0.41	0.12	0.07	2.1	–	places 0.39, 4.2 options, 0.61 sourced	21 s	$0.0020
	Agent	12	0.53	0.64	0.22	1.9	0.42	places 0.68, 3.8 options, 0.98 sourced	75 s	$0.0150
	Agent, propose fix	12	0.62	0.51	0.15	1.5	1.00	places 0.88, 4.6 options, 0.98 sourced	85 s	$0.0148
develop	RAG	13	0.47	0.12	0.12	0.4	–	obstacles 0.44, levers 0.42, asks 0.62	24 s	$0.0021
	Agent	13	0.70	0.74	0.56	1.4	0.92	obstacles 0.68, levers 0.69, asks 0.69	63 s	$0.0114

"Agent, propose fix": the propose guide rewritten (the place is not fixed → propose; work from the comprehensive plan's growth passages in 4–6 turns) and the last turn forced to `answer_ready`, so a run that reaches the step budget still picks its mode.

2.8 Verdict sets with the mode prompt (regression)

Set	Prompt	n	Verdict	Key points	Evidence seen	Unsupported	Mode "verdict"	Latency	Cost / question
14 cases	before (verdict format)	28	1.00	0.77	0.88	1.1	–	31 s	$0.0071
	with modes	18	1.00	0.73	0.92	1.4	0.67	45 s	$0.0068
MÖD	before (verdict format)	200	0.70	0.48	0.70	2.0	–	43 s	$0.0086
	with modes	50	0.70	0.49	0.71	1.9	0.84	64 s	$0.0085

Same cases, agent medium / medium, Luna judge as in the matrix above. The mode runs used CPU embeddings, hence part of the extra latency.

2.9 Optimisation round (2026-10-21)

Goal: improve the agent on the four uses (verdict, explain, propose, develop), in order of importance accuracy, usefulness, cost, time, with cheap models (budget $3.50). Dev set: the 52 cases of the 14-case set, the 10 seeds and

the 28 generated cases, one pass per configuration, graded by MiMo v2.6 Flash in the run and again by GPT-6 Luna (`agentic.rejudge`), so that a gain counts only if both model families see it. Before the runs, the benchmark was fixed: the judge now gets every cited source in full (40 × 3,000 characters instead of 15 × 1,500: a quarter of the answers cited more than 15 sources, whose facts were counted as unsupported); the 9 of the 14 cases whose question also asks "how does it work" or "what would it take" accept explain / develop as mode (`modes`); 3 reference points the asker does not need were dropped (`REVIEW`).

Configuration	n	Key points MiMo judge	Key points Luna judge	Unsupported MiMo / Luna	Evidence seen	Answer length	Median latency	Cost / question
Luna, prompt before 2026-10-21	52	0.69	0.66	0.5 / 1.5	0.79	4,952 chars	53 s	$0.0091
Luna, + recipe per mode and site checklist	52	0.69	–	0.7 / –	0.70	5,012 chars	57 s	$0.0096
MiMo v2.6 Flash as the agent	50	0.79	0.76	1.8 / 7.5	0.72	6,677 chars	124 s	$0.0160
Luna, + "complete answer" (adopted)	52	0.71	0.72	0.5 / 2.0	0.77	5,315 chars	64 s	$0.0098

- **Recipe per mode + site checklist** (choose the mode in the first turn; for explain, read the kommun's page in full; grep the place's constraints: strandskydd, farmland, flood, noise, risk, radon…): key points unchanged (−0.00, 90 % CI −0.05 to +0.04), less of the expected evidence read (−0.09): rejected. More instructions on how to explore do not move the scores.
- **MiMo v2.6 Flash as the agent**: more key points under both judges (+0.10 MiMo, +0.09 Luna, CI +0.04 to +0.15), but 7.5 unsupported claims per answer for the Luna judge (Luna as the agent: 1.4), answers 2.3× slower (median ≈ 2 min, 11k reasoning tokens) and twice the cost: rejected (accuracy first). It read less of the evidence than Luna yet covered more points: Luna's losses are in writing the answer, not in finding the sources.
- **"Complete answer"** (the answer instruction "prefer fewer, well-sourced points" replaced by "include every rule, condition, step, fee, deadline and site constraint from your tool results, each with its source; silent on what they do not say"): +0.03 key points with the MiMo judge (CI −0.00 to +0.06), +0.05 with the Luna judge (CI +0.02 to +0.09), gains on explain / propose / develop, verdict unchanged; same latency and cost; unsupported claims equal for MiMo, +0.5 for Luna (CI −0.0 to +1.0). **Adopted** as the default prompt.
- Not tried: GPT-6 Sol (≈ 20× Luna's price per token, ≈ $0.10 per answer) and MiMo v2.6 Pro (≈ 4×): MiMo Flash showed that a stronger writer covers more but invents more, and the budget did not allow measuring them properly.

Two runs were cut by the machine (memory) and finished from their saved results; latencies are with the query embedding on the CPU. One pass per configuration: differences below ~0.05 are within the noise of a single pass.

2.10 Bug round (2026-10-22)

The scores of the previous sections hid tool failures: they show what the answer covers, not what the agent tried and did not get. Reading the tool calls of the 1,108 stored agent answers found these bugs (fixed, with tests in `tests/test_tools.py`, `tests/test_agent_images.py`):

- `read` **failed on the id as printed**: tool outputs tag chunks `[c:<id>]` and the agent passed `c:<id>` back; 1,845 reads returned nothing, in 57 % of the answers, and 88 % of them were never retried, so the agent answered without the passage it had chosen to read. `read` now takes the id in every form the model writes (with `c:` or brackets, a doc id, a truncated id).
- **Statute chapters filed under the previous chapter**: a chapter heading wrapped onto two lines, "10 Kap." with a capital K, a bare "14 kap." line: MB 4 kap. (national interests), JB 7 / 10 / 11 kap., FBL 1 / 14 kap., MPF 4 kap., PBF 7 kap. had wrong chapter headers, so `law_section` did not find them (~70 calls returned nothing). The text and the chunk ids were right: headers fixed and the affected chunks re-embedded.

- **Municipality-filtered queries scanned the whole table** (`kind != 'law'` does not use the index): ~0.7 s per query, ~2.3 s per answer. `grep` **without a municipality** ran a regex over the whole corpus (12–28 s); it now needs a municipality or the statutes.
- **Every image was sent again on each later turn** (plan map crops, PDF pages): it now goes with the next model call only, and the agent is told to write down what it reads.
- **Miscopied citation ids** (~1 % of the cited ids, wrong after the first characters) left the claim without a source in the chat and for the judge: they now resolve to the chunk shown to the agent with the longest common prefix.
- **Garbled titles** (UTF-8 read as Latin-1 by the crawler: "fÃ¶r") on ~285 documents of 15 municipalities: repaired at extraction, and these municipalities crawled again (their links were scored on garbled words).

Same 52 dev cases, before and after (MiMo judge in the run, Luna judge with `agentic.rejudge`):

Run	n	Failed reads per answer	Empty law_section per answer	Key points MiMo / Luna	Unsupported MiMo / Luna	Evidence seen	Median latency	Median first token	Cost / question
Before the fixes (2026-10-21, 1 pass)	52	2.96	0.04	0.71 / 0.72	0.48 / 1.96	0.77	64 s	54 s	$0.0098
After the fixes (2026-10-22, 3 passes)	155	0.32	0.02	0.71 / 0.69	0.48 / 1.45	0.76	39 s	25 s	$0.0091

The failures are gone and the agent is faster (median answer 64 → 39 s, first token 54 → 25 s) and better grounded (unsupported claims for the Luna judge 1.96 → 1.45 per answer), at the same cost. Key points did not move: the agent made up for the failed calls with more searches, so the bugs cost time and grounding rather than coverage. The results of sections 2.1–2.9 were measured with these bugs: their latencies are ~20–30 s too high and their unsupported claims too high; comparisons between configurations stay valid (all had the same bugs).

Failed tool calls of the run after the fixes (`agentic.health`, printed by every benchmark run since):

Tool	Calls	Failed	Rate	Reasons	
`read`	1,155	49	4%	empty 49	
`law_section`	1,110	3	0%	empty 3	
`search`	789	0	0%	–	
`grep`	398	0	0%	–	
`list_documents`	229	0	0%	–	
`find_kommun`	155	0	0%	–	
`plan_rules`	32	15	47%	empty 15	above threshold
`plans`	21	0	0%	–	
`view_map_area`	3	0	0%	–	
`view_pdf_page`	1	0	0%	–	

`plan_rules` fails on plans whose map was not crawled or has no text layer (no provision extracted: `no_data`): a data gap, not a tool bug.

2.11 Coverage of the municipalities' plans

All the benchmarks above are built from documents of the corpus, so a plan the crawl never fetched cannot appear in them and its absence costs nothing in their scores. This benchmark starts outside the corpus: for 29 sampled municipalities (small, medium and large), their own list of plans in force, fetched live (2026-10-22), and for up to 20 listed plans the PDFs the plan page links to (`agentic.coverage`). Score: the share of those plans and PDFs found in the corpus (matched by URL).

Municipality	Plans sampled	At least one PDF in the corpus	All its PDFs	PDFs found
Arvika	5	40%	40%	40%
Färgelanda	20	10%	10%	10%
Hallsberg	20	90%	90%	90%
Haparanda	8	0%	0%	0%
Härjedalen	20	100%	50%	80%
Kalix	6	100%	100%	100%
Kinda	15	100%	0%	44%
Mullsjö	18	100%	100%	100%
Norrtälje	20	40%	35%	33%
Södertälje	10	90%	90%	97%
Vimmerby	20	95%	90%	90%
Örebro	20	100%	0%	73%
Östhammar	20	85%	70%	85%
All	**202**	**76%**	**51%**	**57%**

16 municipalities could not be measured: their list page links to no plan (they publish their plans in a map or search application, out of reach of a crawler), or was unreachable: Bjuv (no plan with a PDF reachable from the list page (map or search application)), Göteborg (list page unreachable), Hallstahammar (only 1 plan(s) with a PDF reachable from the list page), Hässleholm (no plan with a PDF reachable from the list page (map or search application)), Höör (no plan with a PDF reachable from the list page (map or search application)), Laholm (no plan with a PDF reachable from the list page (map or search application)), Lerum (no plan with a PDF reachable from the list page (map or search application)), Munkedal (only 1 plan(s) with a PDF reachable from the list page), Mölndal (no plan with a PDF reachable from the list page (map or search application)), Ockelbo (no plan with a PDF reachable from the list page (map or search application)), Sjöbo (no plan with a PDF reachable from the list page (map or search application)), Solna (only 1 plan(s) with a PDF reachable from the list page), Sunne (only 2 plan(s) with a PDF reachable from the list page), Timrå (no plan with a PDF reachable from the list page (map or search application)), Tjörn (no plan with a PDF reachable from the list page (map or search application)), Värmdö (no plan with a PDF reachable from the list page (map or search application)). For them the corpus has at most the plans linked elsewhere on the site. File: `coverage-20261025.json`.

Why the 346 missing PDFs are missing, read from each municipality's crawl record:

- **171 PDFs**: its plan page was never crawled, though the list page was (most likely one link past the crawl's depth limit). Haparanda 103, Norrtälje 59, Östhammar 7, Södertälje 2.
- **164 PDFs**: linked from a crawled page but not taken: a link named after the place or the step ("Lagakraftbevis", "Undersökning om betydande miljöpåverkan") has no plan word (crawl.take_pdf_link). Kinda 100, Örebro 20, Färgelanda 18, Härjedalen 17, Vimmerby 3, Östhammar 3, Arvika 1, Hallsberg 1, Norrtälje 1.
- **10 PDFs**: a copy of a document kept under another URL (corpus.extract keeps one per content; the score matches URLs). Norrtälje 3, Arvika 2, Vimmerby 2, Hallsberg 1, Örebro 1, Östhammar 1.
- **1 PDF**: dropped by the PDF filter. Norrtälje 1.

Leads, by expected gain: (1) follow the plan pages of a plan list past the depth limit (the pages under the list's path); (2) on a plan page or a plan list, take every PDF link that is not off topic, whatever its name; (3) count a copy of a kept document as found (match the file, not only the URL). The 11 of 29 sampled municipalities that publish their plans only in a map or search application need another source: Lantmäteriet's national access to digital detailed plans or the municipality's plan layers (WMS / WFS), which also give the plan boundaries (address → plan); otherwise, a browser agent driving the map application, used as a crawler: once per municipality it lists the plans and their document links, which the crawler then downloads (a one-off cost per municipality; at answer time it would be slow, costly and fragile).

2.12 Document filters in the agent tools

The larger corpus makes a kommun's search results a mix of plans, comprehensive plans, permit pages and administration. `search`, `grep` and `list_documents` now take document types and a title filter, applied before ranking; `grep` can list the documents that match instead of the passages, and `list_documents` pages through long lists. The agent is told about them in three lines of its tool guide. Same dev set and protocol as section 2.10 (52 cases × 3 passes, MiMo judge in the run, Luna re-judging), same corpus (2026-10-22), so the difference is the tools and those three lines.

Run	n	Key points MiMo / Luna	Unsupported MiMo / Luna	Evidence seen	Median latency	Cost / question	Answers using a new filter	grep calls with no match
Before (toolfix, 2026-10-22)	155	0.71 / 0.69	0.48 / 1.45	0.76	39 s	$0.0091	0%	1%
With the document filters (2026-10-24)	153	0.73 / (41 rejudged)	0.49 / (41 rejudged)	0.80	38 s	$0.0090	100%	5%

Key points, difference per case (after − before, mean ± standard error): MiMo judge +0.026 ± 0.015 (52 cases); Luna judge +0.065 ± 0.028 (30 cases).

The agent used the new filters in every answer (54 % of its searches, nearly all its greps and document lists), made 2.1 fewer tool calls per answer and covered slightly more key points: +0.026 ± 0.015 per case for the MiMo judge (explain / propose / develop seeds +0.08, generated cases +0.02, the 14 cases unchanged) and +0.065 ± 0.028 for Luna on the 41 answers it re-judged (30 cases) before the OpenRouter credits ran out; 3 of the 156 answers were lost the same way. Unsupported claims and cost did not move. The whole answer is ~1.5 s shorter but starts ~3 s later (first token 26 → 29 s). Narrow title filters return nothing more often (5 % of greps, 1 % before). Measured on the 2026-10-22 corpus, where a municipality has a few hundred documents; on the national corpus, five times larger, the filters should matter more (not measured yet). Kept.

3. Reading the results

- **The search strategy separates the systems, not the volume read.** The RAG has 20 slots for 1–2 expected passages, but one query. On the MÖD cases every expected passage is a statute section, among several thousand, and the question describes facts (a house by a lake, a barn on farmland) rather than the rule: the rewrite rarely reaches the deciding section (evidence 0.12). The agent reasons first (“this is shoreline protection: exemption and its special reasons”) and then fetches the section.
- **Where the RAG is enough.** On the 14 case studies (short questions, well-covered municipalities, answers in one municipal page or a well-known section) it reaches the verdict every time, with the best grounding and half the latency; it still covers 0.18 fewer key points than the agent.
- **Verdict calibration.** The RAG is right more often than the agent on refused projects (53 % vs 39 %) but much less on granted ones (32 % vs 79 %): missing the conditions and exceptions that make a project possible, it is more negative rather than better calibrated. The agent's “with conditions” on refusals seems to come from treating the conditions it finds as requirements to meet rather than as blockers.
- **Exploration effort**: at *low* the agent makes ~12 tool calls per question, at *medium* ~20–26, better targeted; it then sees and cites much more of the expected evidence. More exploration also finds more conditions, hence a slightly lower verdict score on MÖD (0.75 → 0.70) despite better evidence.
- **Grounding is not quality on its own.** The RAG has the highest grounding and the lowest key-point coverage: the measure says that stated claims are supported, not that the important ones are there. Read it together with key points and evidence.
- **Difficulty.** The MÖD and plan-map cases separate the systems; the 14 case studies saturate on the verdict and only show differences in key points and evidence.

4. Recommendations

- **Default app setting**: the agent, *medium* exploration and *medium* answer; *low* exploration as a fast mode (≈ 26 s). A *high* answer is not worth its cost.
- **The classic RAG is not a substitute** for feasibility questions beyond the simplest ones; it could serve where speed matters more than completeness (a first quick answer while the agent explores), to be evaluated as such.
- **Verdict prompt**: state that an unmet substantive condition (productive farmland without an essential public interest, shoreline protection without a special reason, a project that requires a detailed plan) makes the project "not feasible as described", and measure it on the refused MÖD cases.
- **Map reading**: keep `view_map_area`; make it cheaper (one crop instead of crop + page, smaller images) since it doubles the time on plan questions.
- **Benchmark**: run the MÖD sample (50 cases, 2 passes), the plan-map set and the explain / propose / develop sets for every prompt or model change; next, multi-turn cases (iterating on a project with the user), which no set measures yet.
- **Propose**: the fix makes the agent recognise every propose question and name more of the designated places, but it reads less of the expected evidence than before; keep the step budget, and check the cost of the extra places it names.
- **Judge**: have a domain expert review a sample of scores and of plan-map references.
- **Prompt** (section 2.9): the "complete answer" instruction is the default since 2026-10-21; more instructions on how to explore did not help, and MiMo Flash as the agent invents too much.

Next steps (recommended order, 2026-10-21)

1. **Confirm the adopted prompt** ("complete answer"): a second pass on the 52 dev cases, and the MÖD sample and plankarta set, which the optimisation round did not run (≈ $1). A +0.05 gain from one pass is close to the noise.
2. **Check step on the draft answer**: before streaming, check each claim against the chunk it cites and drop or flag what the chunk does not state, to bring the unsupported claims down (≈ 1.5–2 per answer for the Luna judge) without losing the coverage the "complete answer" instruction gained. Measure with both judges and 2 passes.
3. **A stronger model for the writing only**: the gains are in the answer, not in the exploration; test MiMo v2.6 Pro or GPT-6 Sol on the final turn only (Luna keeps exploring), with the check step, on a subset first: Sol costs ≈ 20× Luna per token.
4. **Expert review**: a Swedish planning expert scores a sample of answers and reviews the generated references (key points written by an LLM, reviewed only by the developers).
5. **Benchmark coverage**: "I have this land, what could I do with it?" (place fixed, project open); multi-turn cases (iterating on a project with the user); the localisation work as a `locate` task in the same case format, after the merge.
6. **Latency**: two of 52 answers waited over 6 minutes on a stalled provider stream: retry a stream that stalls, and bring propose (≈ 80 s) closer to the other modes.

Appendix: reproduce

`python -m agentic.bench --systems agent --store disk --explore-effort medium --effort medium --cases-file cases_caselaw.jsonl --sample 50 --repeats 2 --workers 3` (MÖD; `--systems rag` for the classic RAG; `--cases-file cases_plans.jsonl` for the plankarta cases, `--disable-tools view_map_area` for the ablations) · `python -m agentic.grounding <results>` · `python -m agentic.report`. Total cost of the answers in this report: ~$5.64 (judges and case generation excluded). Result files used: bench-20261007-211220-agent-medium-explore-low.json, bench-20261007-214122-agent-medium-explore-low-cases_caselaw.json, bench-20261007-215229-agent-medium-explore-medium.json, bench-20261007-221849-agent-medium-explore-medium-cases_caselaw.json, bench-20261007-222545-agent-medium-explore-medium-cases_caselaw.json, bench-20261007-223027-agent-high-explore-medium.json, bench-20261007-225332-agent-high-explore-medium.json, bench-20261007-224026-agent-

high-explore-medium-cases_caselaw.json, bench-20261007-230229-agent-high-explore-medium-cases_caselaw.json, bench-20261008-134746-agent-medium-explore-medium-cases_plans-no-view_map_area-view_pdf_page.json, bench-20261008-141019-agent-medium-explore-medium-cases_plans-no-view_map_area.json, bench-20261008-143710-agent-medium-explore-medium-cases_plans.json, bench-20261017-133642-rag-medium-explore-low.json, bench-20261017-134133-rag-medium-explore-low-cases_caselaw.json, bench-20261017-140205-rag-medium-explore-low-cases_plans.json.